AI-Training Collective Communication in gem5 Garnet

Router-Level Multicast, Express-Link Bypass, and Tensor AllReduce

Haru-LCY and Shealligh

August 10, 2026

Abstract

Distributed GPU training moves tensors among devices over and over again through collectives such as broadcast and all-reduce. When the same data is replicated as independent unicast packets, shared links carry redundant traffic; when routes span many hops, synchronization latency grows. This project asks whether an interconnection network that is aware of collective structure can move AI-training data more efficiently. Within gem5 Garnet, we implement three mechanisms: router-level tree multicast that shares packet prefixes, physical express-link bypass that shortens selected routes, and streaming in-network tensor all-reduce that merges contributions before a single result broadcast. A trace compiler ties these mechanisms to an executed eight-H100 NCCL microbenchmark. All 10,164 runs across the multicast, bypass, interaction, and bypass-optimization matrices pass their completion and statistic checks. Over 162 paired cases, tree multicast reduces internal-link flit traffic by 39.51% on average and delivers a 3.588× median completion-latency speedup, though 10 cases regress in throughput. For data traffic on distance-scaled diagonal links, congestion-aware bypass achieves a 1.080× geometric-mean average-latency speedup and a 1.086× p95-latency speedup over Mesh across 384 paired cases, compared with 1.068× and 1.072× for static bypass. Tensor streaming shrinks the all-reduce trace window from 80,638,000 to 17,054,500 ticks, a 4.728× gain over scalar-lane replay. The bypass effect is workload dependent: speedup reaches 1.213× for transpose traffic and 1.266× at offered load 0.7, and settles to 1.006× at load 0.05.

1 Introduction

1.1 The training step is a communication pipeline

Modern AI models are trained across many GPUs. Data parallelism must reduce gradients across workers, while model and tensor parallelism additionally exchange parameters, activations, and partial results. These exchanges are organized as collectives—most notably broadcast and all-reduce—and they typically sit on the synchronization path of every training step. As GPU computation grows faster, the cost of moving the same tensor through the interconnect can rise to become a first-order concern.

Figure 1 makes this motivation concrete. A single training step alternates between local GPU work and a synchronization phase: each rank computes a gradient shard, the ranks exchange or reduce that shard, and only then can the next step safely consume the result. The figure is a conceptual mapping that shows where a router-level design enters the stack.

The conventional network we use as a baseline is oblivious to this collective structure. A broadcast to N destinations degenerates into N independent unicast packets, so shared path segments carry duplicate flits. Deterministic XY routing likewise forces every packet through ordinary Mesh links, even when a well-placed shortcut could eliminate several hops. And decomposing an all-reduce tensor into thousands of independent scalar operations throws away the streaming and aggregation opportunities that the original tensor makes available.

Our project draws on the collective-aware data movement found in modern multi-GPU systems and asks how much of that efficiency can be recovered inside Garnet. The guiding idea is to preserve the semantics of an AI collective while cutting redundant packets and hops:

- **Tree multicast** injects a single logical packet and replicates it only where destination paths branch, so common route prefixes are shared.
- **Physical bypass links** grant selected packets an express hop across intermediate Routers, subject to routing safety and wire cost.
- **Tensor all-reduce** keeps a tensor as a multi-flit collective, merges eight rank contributions inside the network, and broadcasts a single completed result.

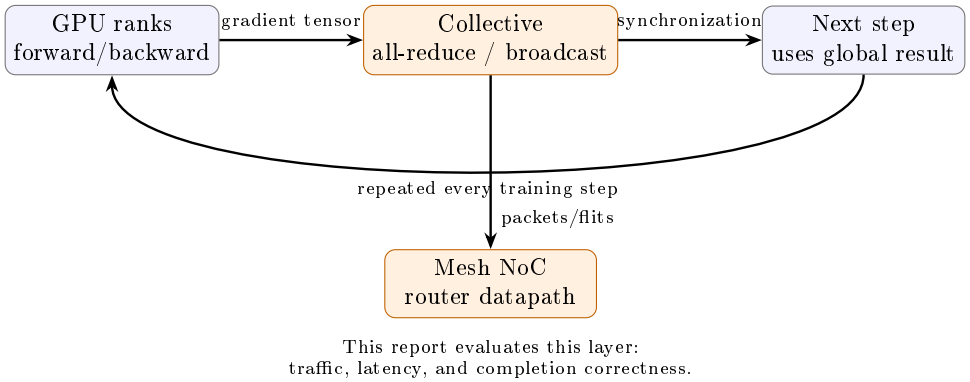

Figure 1: From a modern multi-GPU training step to the network mechanisms evaluated in this report. The communication phase is on the critical path when computation cannot proceed until the collective completes.

A trace compiler and replay path then translate broadcast and all-reduce events captured from a real eight-H100 NCCL microbenchmark into these network operations. The evaluation follows three corresponding questions: does multicast reduce duplicated network traffic, when do bypass latency gains survive a distance-scaled wire model, and does tensor streaming avoid the cost of scalar serialization?

For Lab 4, the primary submission is **Topic 3: Microarchitecture**, which implements both required technologies, Multicast and Bypass. Trace replay is additional **Topic 4** work, and tensor all-reduce reinforces the shared AI-training narrative. The scope is *AI-training collective communication* at the network layer, and the results quantify the behavior of these router-level mechanisms.

2 Architecture and Implementation

2.1 Baseline and completion contract

The baseline is a Garnet Mesh with deterministic XY routing. For a logical multicast to N destinations, `naive_unicast` emits one standard physical packet per remote destination. Each packet traverses the normal pipeline: input VC, route computation, VC allocation, switch allocation, crossbar, output VC, and credit stages.

A logical operation completes only once every expected destination has received exactly one complete packet with the correct collective ID, round/lane ID, and value. Missing, duplicate, unexpected, wrong-round, wrong-lane, or wrong-value delivery is fatal, and reaching a tick limit does not count as completion.

2.2 Router-level tree multicast

In `tree_multicast`, the Network Interface injects a single packet whose flits carry a 64-bit destination bitmap. At each Router, the datapath prunes destinations that have already been served, computes the union of required output branches, produces a local copy only where one is selected, and maintains branch state across head, body, and tail flits. Fanout is atomic: a flit advances only when every selected branch has an available output VC and credit. This keeps multi-flit delivery safe under backpressure, but it can also force a free branch to wait on a blocked one. Because the bitmap is 64 bits wide, one multicast domain is limited to 64 Routers.

The multicast datapath is a dedicated Router path. It runs its own output-VC and credit checks before placing copies into output units, and so it does not traverse exactly the same switch-allocation and crossbar path as baseline unicast. For this reason, internal-link traffic reduction is the cleanest structural comparison between the two. The latency speedup, by contrast, reflects the complete implemented mechanism—including this fast path—and should not be attributed to tree sharing alone.

2.3 Physical bypass links and routing

`Mesh_Bypass` adds real bidirectional Garnet `IntLinks`, and we evaluate both diagonal and stride placement families. A generated all-pairs oracle picks at most one source express hop followed by deterministic XY, and its channel-dependency graph is verified to be a DAG. Two link models bracket the timing assumption:

- **optimistic**: a shortcut costs a single cycle, an upper bound on its benefit;

- **distance-scaled**: latency and serialization reflect the Manhattan wire span.

The cost record reports the extra links, a total wire-length proxy, the maximum Router radix, the added input ports, and a buffer-slot proxy.

The optimized policy adds bounded congestion lookahead. On an unordered data VNet, the source compares the number of free downstream VCs on the oracle's express port against those on the ordinary first XY port. The shortcut's reverse credit path additionally supplies a coarse occupancy signal for the landing Router's next output: a busy landing port triggers an XY fallback, whereas an idle one lets the source's express-versus-XY free-VC pressure and the packet length settle the choice. A compact epoch monitor updates one hot-destination bit per Router every $16N$ injected data packets, marking any destination that exceeds four times the uniform share as skewed. For such many-to-one destinations, one-flit packets fall back to XY; otherwise they require a two-VC express advantage. Four-flit packets may take an idle express/XY tie in order to capture the saved pipeline hop, while longer packets still require the two-VC advantage. Packets longer than 32 flits also use XY, because a long wormhole reservation could otherwise monopolize a shortcut; this packet-length cutoff is a command-line parameter. Ordered VNet behavior remains static. Because the decision is made only at the source and every suffix is plain XY, the ordinary XY dependency graph stays acyclic: express channels contribute only outgoing dependencies into that graph and so can never close a cycle.

For multicast, a cost-aware policy takes a source express hop only when the destination's ordinary XY path shares no edge with any other destination's path, which avoids destroying useful tree sharing. A directed 2×2 diagonal case demonstrates that the joint path can indeed use a shortcut, though full-group collectives more often preserve their ordinary paths instead.

Figure 2 summarizes the joint design on a three-destination example. Replicated unicast would inject three packets, whereas tree mode injects a single bitmap packet. The isolated destination at Router 0 can take the diagonal express link, while destinations 3 and 15 keep their shared ordinary prefix and branch only inside the network.

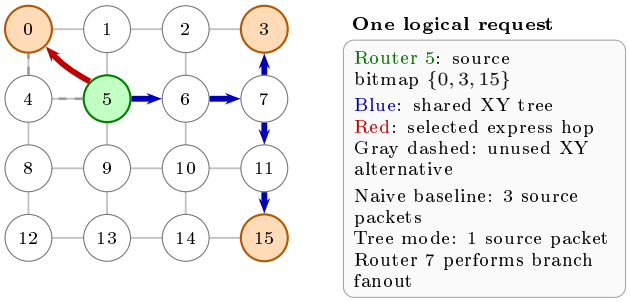

Figure 2: Integrated multicast and bypass example on a 4×4 Mesh. Cost-aware routing uses an express hop for an isolated destination but keeps the shared XY prefix for the other destinations. The diagram shows logical structure; link timing is evaluated separately by the two wire models.

2.4 Trace compiler and tensor all-reduce

The checked-in source trace was captured by running a controlled collective microbenchmark on eight NVIDIA H100 80GB HBM3 GPUs with PyTorch 2.8.0+cu128, CUDA 12.8, and NCCL 2.27.3. Its 48 events comprise 15 measured broadcasts and 15 measured all-reduces at 1, 4, and 16 MiB. Byte volume and relative release time are explicitly scaled by 1/1024, and a flit is 16 bytes.

Broadcast events lower to variable-length tree multicast. The legacy all-reduce lowering serializes each tensor flit as an independent scalar lane; the tensor lowering instead builds 15 multi-flit logical requests, each with 64–1,024 flits per rank and one to four chunks. Router state is keyed by collective, chunk, and lane, and once all eight contributions have merged to the known sum of 36, the result is tree-broadcast. Backpressure gates vary the link latency, Router latency, and number of outstanding requests. The current accumulator uses an unbounded software map and does not yet model a finite entry table or an adder pipeline.

Figure 3 contrasts the two trace lowerings. Both perform the same total rank contributions and Router flits, but scalar replay turns every tensor flit into a separate logical operation, whereas tensor mode keeps multi-flit requests intact through reduction and result broadcast.

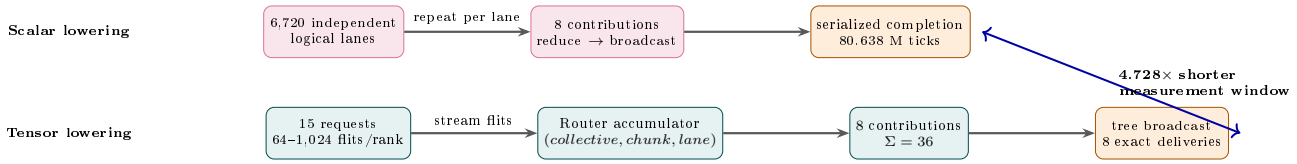

Figure 3: Scalar-lane replay versus the implemented tensor all-reduce path. Tensor mode maintains request/chunk/lane identity while Routers merge eight rank contributions, verify the sum, and multicast the result.

3 Experimental Method

The simulator binary is `build/Garnet_standalone/gem5.opt`, built from gem5 v23.0.0.1 at revision `77fcf26d57`; the full hash is recorded in the manifest. The host exposes 128 logical CPUs, but its cgroup quota is 64 CPU equivalents, so compilation and large sweeps use at most 64 workers.

Table 1: Final quantitative experiment inventory.

Experiment	Raw runs	Pairs/points	Main dimensions
Multicast performance	324	162 pairs	fanout, flits, load, background, seed
Bypass performance (G9)	7,680	6,144 pairs	size, traffic, flits, load, design, seed
Bypass optimization	1,536	768 Mesh pairs	routing mode, traffic, flits, load, seed
Multicast × bypass (G10)	624	416 pairs	mode, wire model, flits, group
Scalar × tensor trace	4	2 pairs	Mesh XY and Mesh Bypass

The multicast performance matrix uses a single 4×4 Mesh with source Router 5. It sweeps destination counts of 4, 8, and 16; packet lengths of 1, 4, and 16 flits; request probabilities of 0.1, 0.5, and 1.0; background rates of 0 and 0.1; and seeds 1, 7, and 17. Each pair measures 100 requests, framed by 20 warmup and 20 cooldown requests.

G9 uses 4×4 and 8×8 Meshes; uniform-random, transpose, bit-complement, and hotspot traffic; 1-, 4-, 16-, and 64-flit packets; 16 offered loads from 0.01 to 3.0 flits/node/cycle; seeds 1, 7, and 17; and all four placement/wire-model combinations. Each run has 1,000 warmup, 100,000 drain, 5,000 measurement, and normally 1,000,000 cooldown cycles; the manifest records 17 cases that required the longer cooldown window.

We retain G9 as a broad topology and wire-model screening study, but it injected all packets into control VNet 0, which has only one buffer per VC and is not a sound proxy for 4–64-flit AI tensor data. The optimization study therefore maps both Mesh and bypass traffic onto data VNet 2 (four buffers per VC) and restricts itself to the hardware-relevant distance-scaled diagonal design. It covers the same two mesh sizes, four traffic patterns, four packet lengths, and seeds 1, 7, and 17, at offered loads 0.05, 0.1, 0.3, and 0.7. The static and adaptive modes each contain 384 paired seed cases and use identical offered source/destination sequences.

For paired latency L, internal-link flits F, and throughput Q, we report

$$S_L = L_{base}/L_{new}, \qquad R_F = 1 - F_{new}/F_{base}, \qquad I_Q = Q_{new}/Q_{base} - 1.$$

All arithmetic means, minima, maxima, and negative cases are preserved in the released CSV/JSON files.

4 Correctness Results

The one-command full gate passes all seven stages: 11 legacy collective cases; 208 paired multicast cases; the 30-request H100 broadcast replay; 14 tensor correctness cases; 32 tensor backpressure cases; the tensor H100 replay; and the paired scalar/tensor replay. The trace tools pass a further 12 unit tests. The bypass baseline-equivalence gate passes on 2×2, 3×3, and 4×4 pairs, comparing 11 statistics in each, and G10 passes all 624 of its 624 interaction runs.

During the new G10 run, 40 cases initially failed only the runner's theoretical Router-flit expectation: the checker counted a plain XY tree even where the cost-aware policy had correctly selected an express edge. The checker now reconstructs the policy from the independently dumped routing oracle. The simulator binary itself was left unchanged, and after this test-only correction all 624 runs pass their exact-delivery, source-flit, Router-flit, physical-packet, and DAG checks.

The current-revision bypass optimization adds another 1,536 passing runs out of 1,536. Its topology dump records whether adaptive admission is enabled, the runner checks that metadata, and both modes draw on the same 19-file source-hash set. Changes in express-link activity exercise the adaptive choice, while packet and flit injection and receipt remain exact.

5 Quantitative Results

5.1 Multicast

Across 162 pairs, the mean internal-link flit reduction is 39.51% (range 16.67–53.13%). Broken down by destination count, it rises to 26.19%, 39.22%, and 53.13% for 4, 8, and 16 destinations respectively. This monotonic trend with fanout is the central evidence that replication is being shared inside the network.

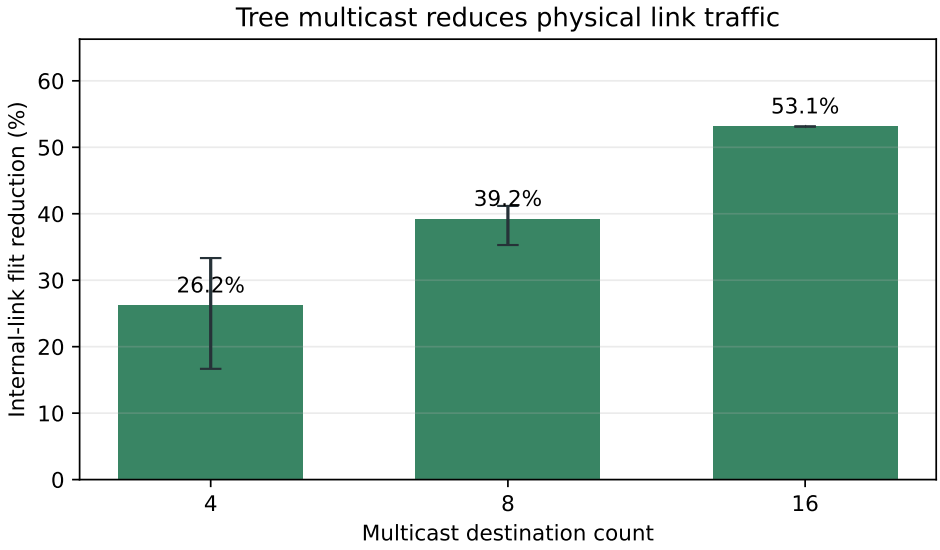

Figure 4: Current-revision multicast internal-link flit reduction. Error bars span observed minima and maxima, not confidence intervals.

Latency speedup has a mean of 4.790×, a median of 3.588×, and a range of 1.200–22.512×, with all 162 values exceeding one. The mean throughput change is +190.89% and the median is +121.66%, though the distribution is skewed by full-group traffic. Ten of the 162 cases regress, all of them at four destinations, with a worst case of −18.24%. Reduced link traffic therefore does not guarantee higher throughput under atomic fanout.

Figure 6 makes this workload dependence explicit. The left panel reports throughput rather than latency, including the observed minima and maxima, while the right panel shows the latency benefit growing as injection pressure rises. The tree is thus not merely a constant-factor optimization: it removes duplicated link work, and that saved work matters most precisely when the network is busy.

Figure 7 shows the mechanism-level trade-off. Larger fanouts move rightward because the tree shares more internal-link flits, yet throughput does not rise monotonically at every point. The negative points are the ten four-destination regressions noted above—exactly why this report keeps link-traffic reduction and end-to-end throughput as separate claims.

5.2 Bypass links

The broad mean conceals important negative evidence. At offered loads of at most 0.1, the distance-scaled diagonal and stride latency speedups are only 0.962× and 0.936×. A 4×4 diagonal placement adds 9 undirected links, a wire proxy of 18, and a maximum Router radix of 8; on 8×8 it adds 49 links and a wire proxy of 98. Stride removes more Router traversals but adds 16/96 links and a wire proxy of 32/192. Hop reduction on its own is therefore not a sufficient hardware argument.

The corrected data-VNet A/B study then optimizes the chosen distance-scaled diagonal design. Because the speedups are ratios with a long saturation tail, Table 3 reports geometric means as its primary aggregate. Congestion

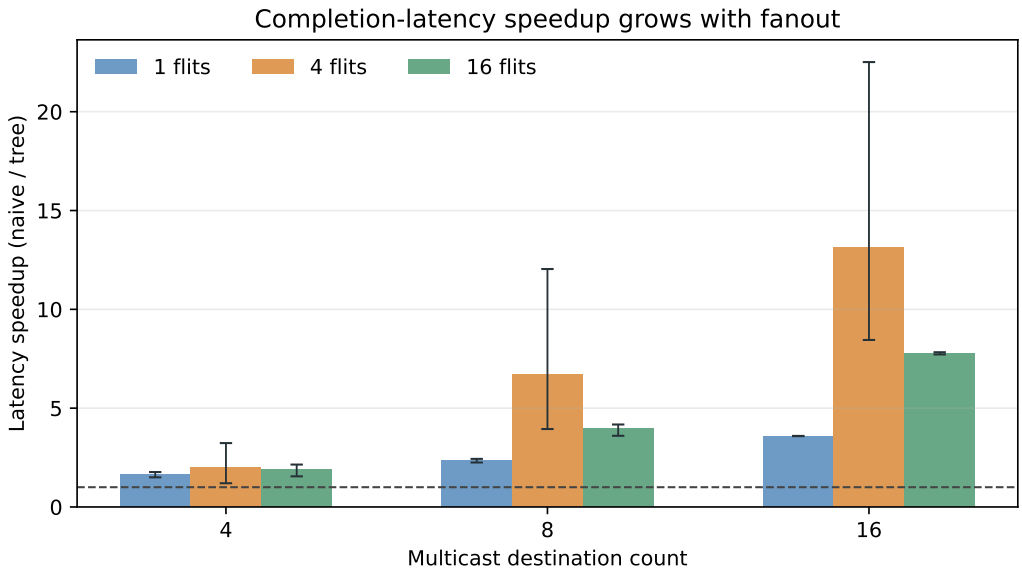

Figure 5: Multicast completion-latency speedup by fanout and packet size.

Table 2: Mean G9 result. Each design contains 1,536 paired seed cases (512 seed-aggregated points).

Design	Latency speedup	Throughput change	Traversal reduction
Diagonal, distance-scaled	1.629×	+6.41%	−2.82%
Diagonal, optimistic	1.723×	+10.61%	−7.03%
Stride, distance-scaled	0.981×	−1.94%	+15.50%
Stride, optimistic	1.103×	+3.68%	+10.10%

admission improves the geometric-mean average-latency speedup by 1.1% relative to static bypass and the p95 by 1.4%, and mean Mesh-relative throughput stays positive at +0.83%. More importantly, it heads off the static bit-complement overload cases: the worst Mesh-relative speedup rises from 0.128× to 0.980×, and regressions exceeding 5% drop from 44 to zero out of 384 cases.

Table 3: Distance-scaled diagonal bypass on data VNet 2 (384 paired seed cases per mode). Latency columns are geometric-mean Mesh/bypass speedups.

Mode	Avg. latency	p95 latency	Mean throughput	$> 5\%$ regressions
Static oracle	1.068×	1.072×	+1.92%	44
Congestion-aware	1.080×	1.086×	+0.83%	0

Figure 9 expands this aggregate into the full adaptive matrix. The 1.0 boundary makes the intended operating region visible: bypass is nearly neutral under light load, while structured traffic and high offered load reveal larger gains. The matrix also shows why a single average would mislead—transpose at 4×4, for instance, is far more sensitive to packet length than the corresponding 8×8 cases.

The tail behavior mirrors the mean. Figure 10 shows that p95 latency tracks the average improvement for transpose and high load while staying close to one for uniform traffic and low load. This matters for a training-style workload: the optimization improves the congestion episodes themselves rather than merely shifting the central tendency.

Finally, Figure 11 separates the mechanism from its outcome. Adaptive routing reduces Router traversals in every traffic family, and the resulting latency speedup follows from whether those saved hops fall on the congested critical path. Physical bypass thus acts as a congestion-relief mechanism, delivering its gains where the network is loaded.

Broken out by traffic pattern, the adaptive speedups are 1.022× for uniform-random, 1.213× for transpose,

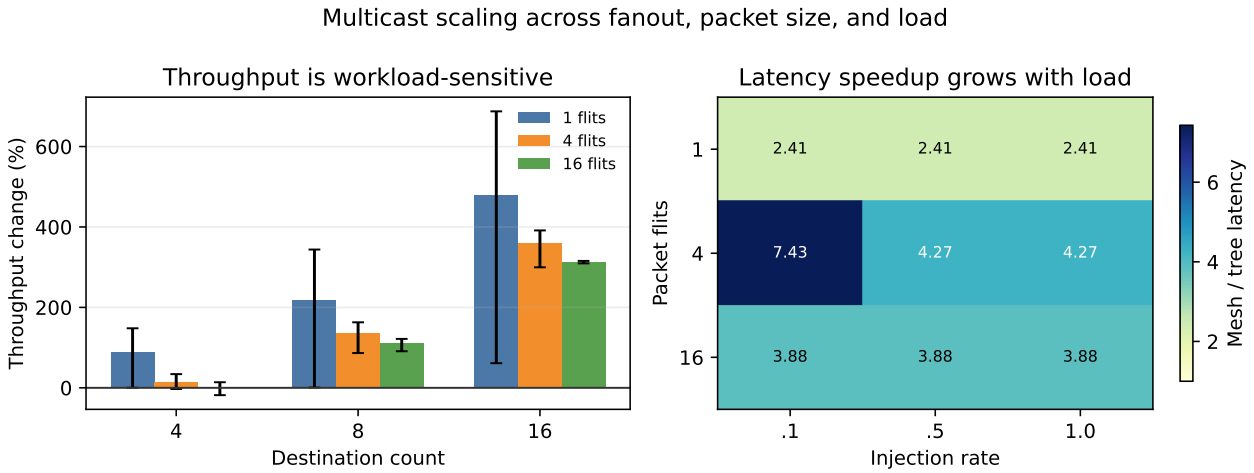

Figure 6: Multicast scaling across destination count, packet size, and load. Bars show mean throughput change with min–max whiskers; the heatmap shows geometric-mean completion-latency speedup across background conditions.

1.044× for bit-complement, and 1.052× for hotspot. By offered load, the speedup is 1.266× at 0.7 and 1.006× at 0.05. The original 4×4 hotspot/load-0.3 weakness is gone: those 12 cases now show a 1.091× geometric-mean speedup with a 1.000× minimum. Eight mild regressions remain across the full matrix, but the worst is 0.980× and none exceeds 5%. The bypass result is thus significant for structured or heavily loaded NoC traffic and intentionally close to neutral when the network is lightly loaded. The adaptive admission policy is essential here: static bypass has a useful mean but can produce pathological individual regressions.

G10 further shows that the interaction between multicast and bypass is workload dependent. Under the distance-scaled model, the mean latency speedup is 0.965× for replicated unicast and 0.988× for tree multicast; the optimistic values are 1.029× and 1.022×. The cost-aware tree uses 85,000 express-link flits across the paired runs, against 357,000 for unicast—evidence that it applies shortcuts selectively rather than forcing every multicast branch onto an express link.

5.3 Real-H100 trace and tensor all-reduce

Both Mesh designs complete all 30 broadcast requests, 6,720 source flits, and 210 destination deliveries in 17,047,500 measurement ticks, with a p95 of 643,500 ticks and a wire-flit distance of 47,040. Express usage is zero, so this is a neutral bypass result.

For tensor all-reduce, both designs complete 15 logical requests, 53,760 rank contributions, 53,760 exact reductions, 53,760 deliveries, and 147,840 Router flits. The tensor window is 17,054,500 ticks; the p50/p95/p99 logical-request latencies are 647,500/2,567,500/2,567,500 ticks; and the peak number of active lanes is two. Reduce and broadcast each contribute 47,040 internal-link flits.

The scalar window is 80,638,000 ticks, so tensor mode yields a 4.728× measurement-window speedup on both Mesh XY and Mesh Bypass. The source and Router flit counts are identical between the two lowerings, so the gain comes purely from representing a tensor as a multi-flit collective rather than serializing 6,720 independent logical lanes. Express usage is again zero for this all-rank convergence-and-broadcast tree.

6 Limitations and Threats to Validity

- Multicast performance uses a single 4×4 source placement, so corner, edge, and center sources are not distinguished.
- The multicast fast path does not share the entire normal unicast switch-allocation and crossbar pipeline, so its latency results blend tree sharing with microarchitectural-path differences.
- Atomic fanout couples branches together, which is why link traffic can fall even as throughput regresses.
- The bypass wire cost is a Garnet latency, serialization, and static-cost proxy—not area, power, repeater, or timing-closure evidence.

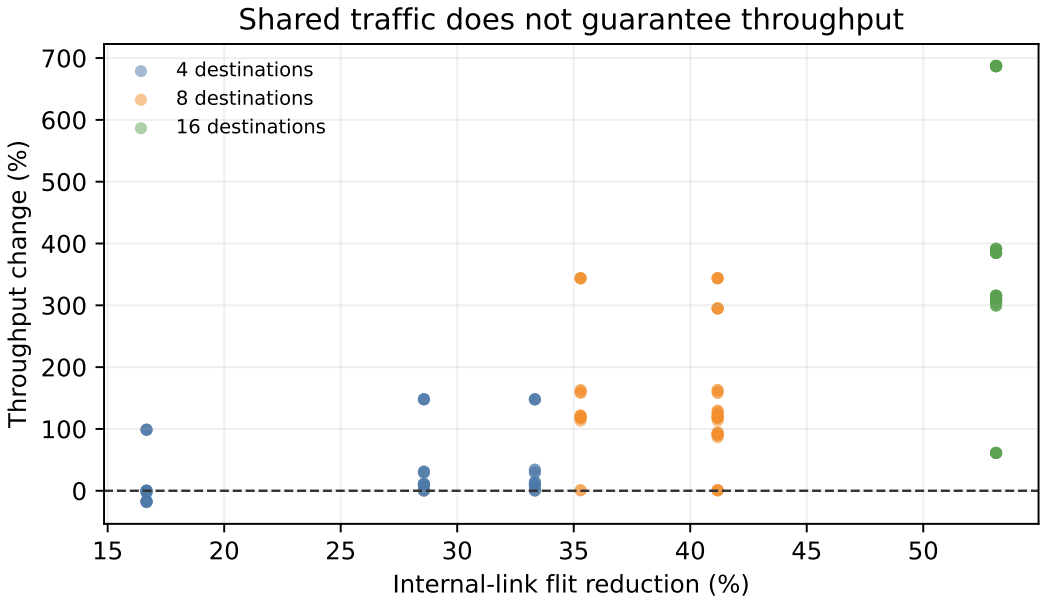

Figure 7: Paired multicast cases: internal-link flit reduction versus throughput change. Color denotes destination count; the dashed line marks unchanged throughput.

- Bypass lookahead reaches only as far as the express landing Router. Eight of the 384 optimized cases remain mildly below Mesh, though none is more than 5% slower. The occupancy and epoch destination-skew signals are modeled without separate propagation-delay, counter-area, or wire-cost accounting.
- The robust policy deliberately routes packets above 32 flits over XY. This removes the long-packet shortcut regressions, but it also leaves 64-flit cases with no bypass benefit; segmentation is left to future work.
- The H100 source is an executed collective microbenchmark rather than a full model trace, and scaling by 1/1024 alters absolute timing and volume.
- Tensor accumulator capacity and arithmetic latency are not modeled as finite hardware resources in the current implementation.
- The results characterize this simulator mechanism; they do not measure NVLink/NVSwitch hardware or end-to-end model throughput.

7 Division of Labor

The commit history gives an auditable two-person split.

- **Haru-LCY** authored the multicast and bypass work. This covers the Lab 4 collective network plumbing; explicit completion-driven collective operations; the replicated-unicast baseline and the router-level tree multicast datapath with destination-bitmap carry and pruning; backpressure-safe multi-flit delivery; and the multicast performance workloads. For bypass, it covers the baseline-equivalence gate, the audited express-link construction and deadlock-checked routing oracle, the wire- and traversal-cost audit, and the correctness, performance, cost-aware, and multicast-interaction gates. It also includes the eight-H100 trace capture, compiler, and replay, and the final optimization arc: hardened routing, the low-load regression fix, epoch-based skewed-destination detection, and the report evaluation and figures.
- **Shealligh** authored the tensor all-reduce work on a dedicated branch: the multi-flit replay contract, compiler, and validator (H1); the in-network tensor reduction and broadcast datapath (H2); the backpressure datapath and H3–H6 gates; the link-latency and credit-conservation statistics; and the full-gate runner. This branch also contributed a multicast-matrix fix that corrected the destination-ordering in `tree_edges()`.

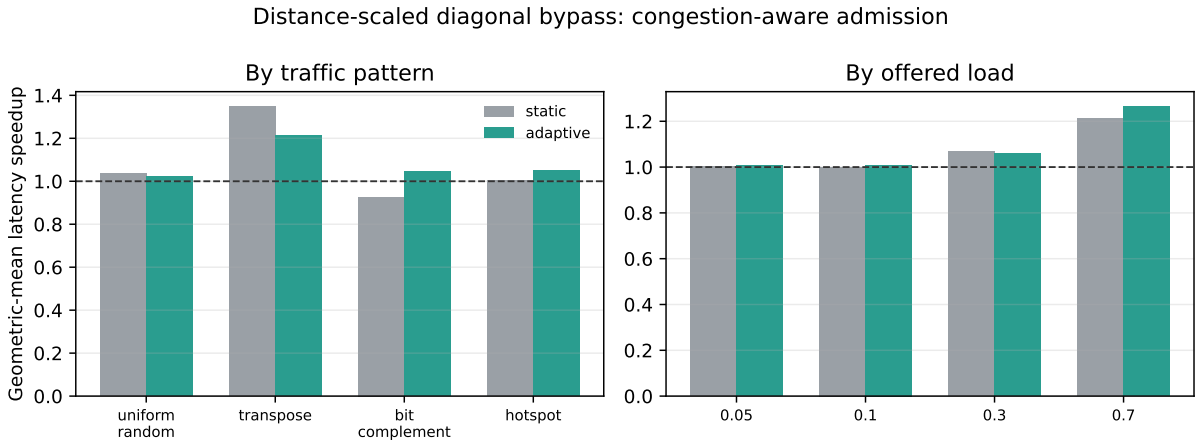

Figure 8: Congestion-aware admission improves the selected physical-wire design across traffic patterns and offered loads while eliminating all regressions larger than 5%.

- **Joint**: branch integration via the tensor-all-reduce merge, regression review, the final quantitative rerun, the limitation analysis, and report verification.

8 Reproducibility

Raw outputs are under `report/raw-results/head-77fcf26d/` and are intentionally git-ignored; optimization outputs are under `report/raw-results/bypass-optimization/`; compact accepted CSV/JSON snapshots are under `report/data/`. The following commands reproduce the principal evidence:

```
python3 tests/gem5/lab4/run_lab4_full_gate.py \
  --gem5 build/Garnet_standalone/gem5.opt --jobs 64
python3 tests/gem5/lab4/run_multicast_performance.py \
  --gem5 build/Garnet_standalone/gem5.opt --jobs 32
python3 tests/gem5/lab4/run_bypass_performance.py \
  --gem5 build/Garnet_standalone/gem5.opt --inj-vnet 0 \
  --no-bypass-adaptive-routing --jobs 64
python3 tests/gem5/lab4/run_bypass_performance.py \
  --gem5 build/Garnet_standalone/gem5.opt \
  --output report/raw-results/bypass-optimization-v7/static-data-vnet \
  --inj-vnet 2 \
  --families diagonal --wire-models distance_scaled \
  --offered-loads 0.05 0.1 0.3 0.7 \
  --no-bypass-adaptive-routing --jobs 64
python3 tests/gem5/lab4/run_bypass_performance.py \
  --gem5 build/Garnet_standalone/gem5.opt \
  --output report/raw-results/bypass-optimization-v7/adaptive-data-vnet \
  --inj-vnet 2 \
  --families diagonal --wire-models distance_scaled \
  --offered-loads 0.05 0.1 0.3 0.7 \
  --bypass-adaptive-routing --bypass-adaptive-max-packet-flits 32 \
  --jobs 64
python3 tests/gem5/lab4/run_multicast_bypass_matrix.py \
  --gem5 build/Garnet_standalone/gem5.opt --jobs 64
python3 tests/gem5/lab4/run_tensor_allreduce_paired.py \
  --gem5 build/Garnet_standalone/gem5.opt
python3 report/scripts/build_final_report_data.py
```

Before writing any summary CSVs and figures, the compact data builder rejects wrong run counts, raw errors,

inconsistent optimization VNet/routing metadata, incomplete multicast measurements, failed full gates, and missing designs. The optimization manifests hash every modified configuration, topology, runner, routing, network, and output-VC source file used by the current binary.

9 Conclusion

Starting from the communication demands of modern multi-GPU training, this project demonstrates three ways to preserve collective structure inside the network: sharing common broadcast paths, skipping selected intermediate Routers, and aggregating tensor contributions before broadcasting the result. It fully satisfies the two-person Topic 3 requirements, since both router-level multicast and physical bypass links are implemented, tested, and compared against appropriate Mesh and unicast baselines. Multicast achieves a 39.51% mean internal-link traffic reduction whose benefit grows with fanout, though 10 throughput regressions and the dedicated fast path qualify the latency result. Bypass is meaningfully effective and workload dependent: adaptive admission reaches 1.080× overall, 1.213× on transpose, and 1.266× at high load, removes the 4×4 hotspot weakness, and cuts regressions larger than 5% from 44 to zero. It delivers its gains on structured and heavily loaded traffic and stays near neutral when the network is light. Finally, the exact replay of broadcast and all-reduce events from a real eight-H100 microbenchmark ties the mechanisms back to AI-training communication, and the tensor path's 4.728× reduction shows that preserving a streaming collective is more efficient than scalar serialization.

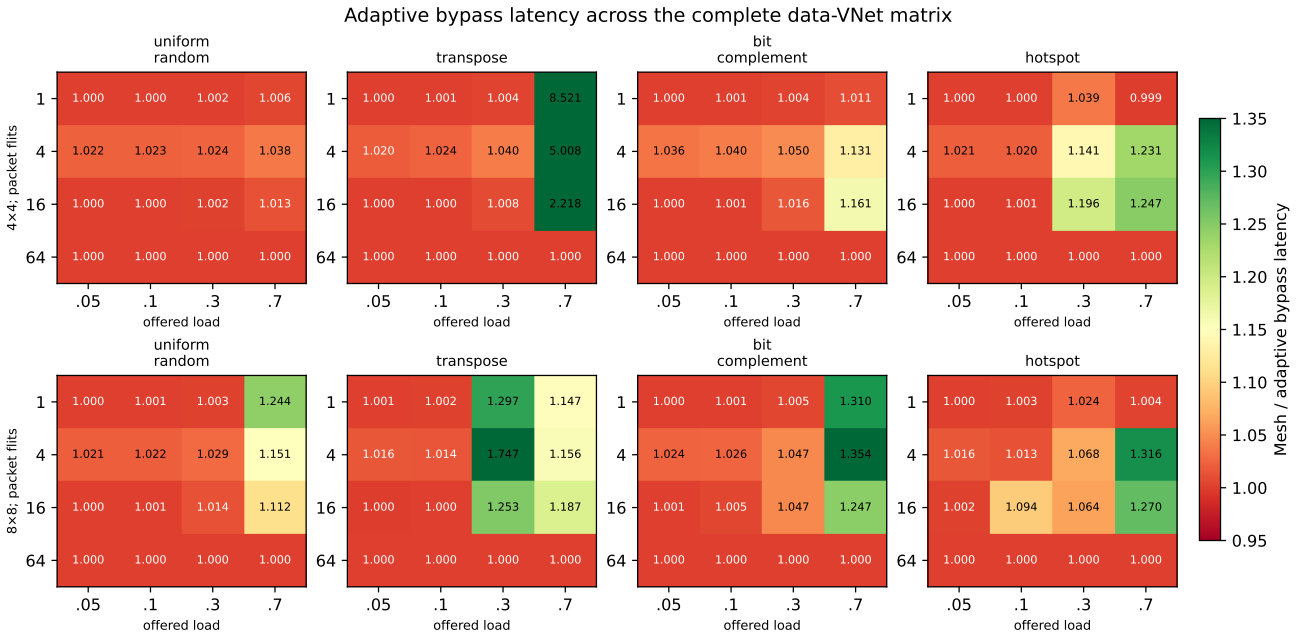

Figure 9: Complete adaptive data-VNet matrix. Each cell is the geometric-mean Mesh/bypass latency speedup over three seeds; rows separate mesh size, columns traffic pattern, and each panel enumerates packet length (rows) and offered load (columns). Values above 1 indicate a bypass benefit.

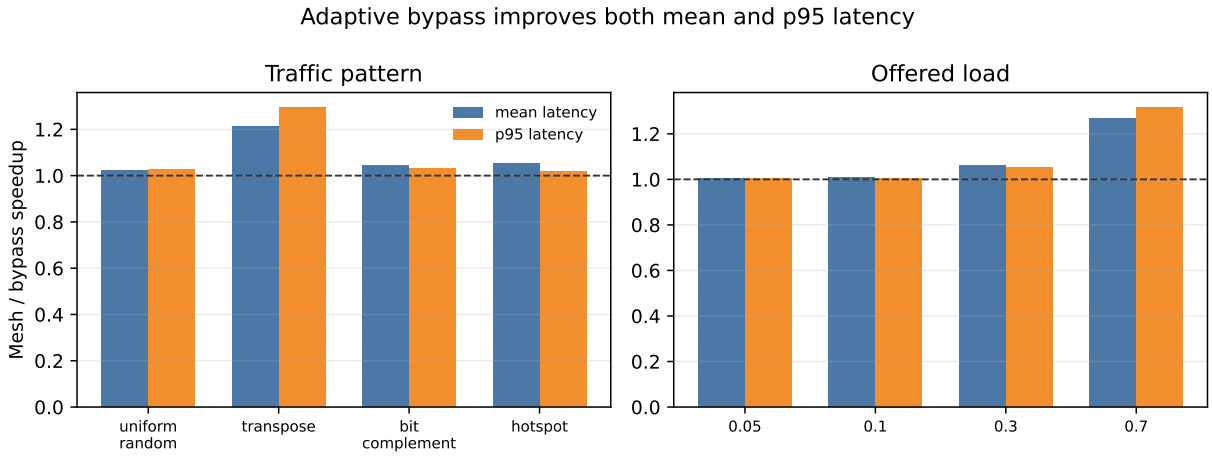

Figure 10: Adaptive bypass latency speedups grouped by traffic pattern and offered load. Bars report geometric means for average latency and p95 latency, exposing both central and tail behavior.

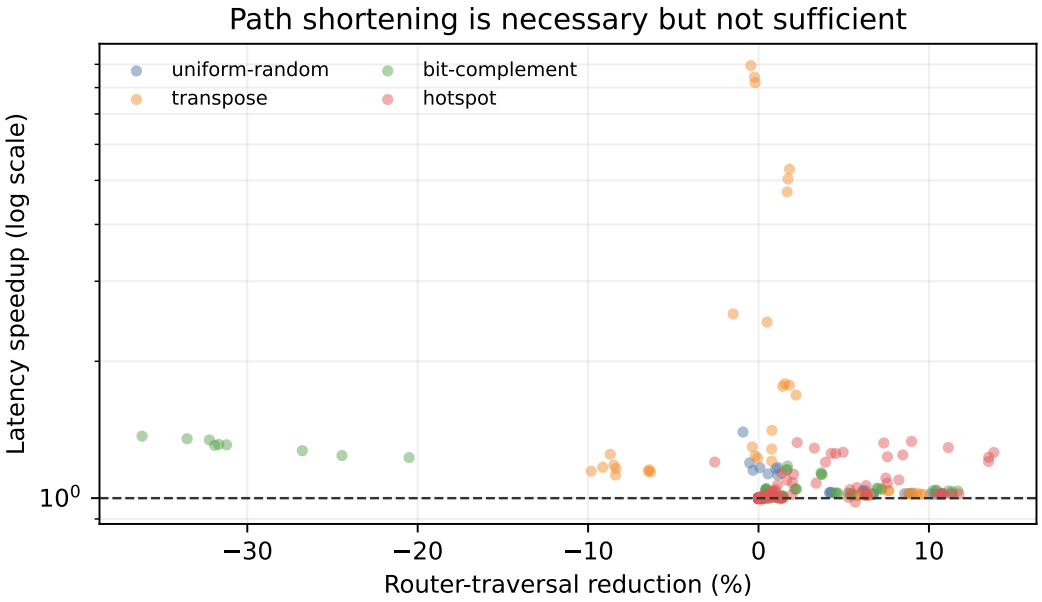

Figure 11: Mechanism-level trade-off for adaptive bypass. Each point is one traffic/size/load/flit aggregate; the x-axis is Router-traversal reduction and the y-axis is Mesh-relative latency speedup. Color identifies traffic.

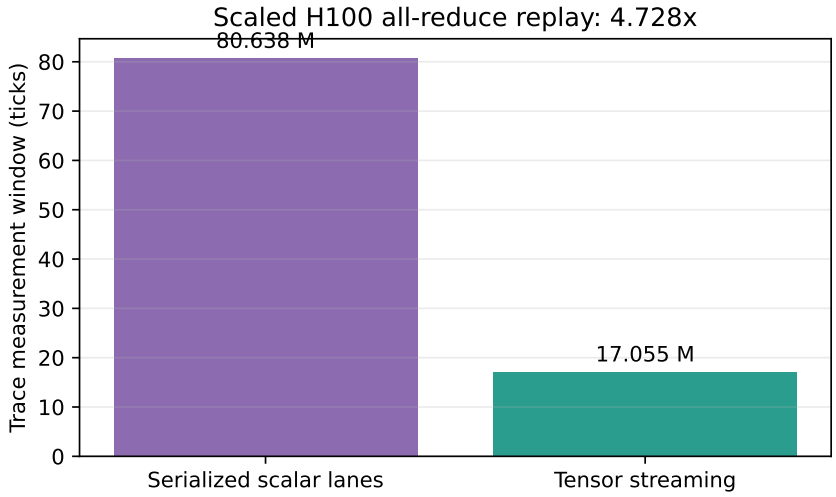

Figure 12: Same scaled H100 all-reduce trace and topology: tensor streaming versus serialized scalar-lane lowering.